

JADAVPUR UNIVERSITY
DEPARTMENT OF PHYSICS

Numerical Integration using the Monte Carlo Simulation

Monte Carlo Integration

$$\int_a^b f(x) dx \approx (b-a) \frac{1}{N} \sum_{i=1}^N f(x_i)$$

Submitted by:
Mohak Das

Supervisor:
Dr. Sahinur Reja

Class: PG-I
Class Roll No: 002520702028
Exam Roll No: PPHD0261034

COMPUTATIONAL PHYSICS LAB

Abstract

This report investigates the application and statistical validity of Monte Carlo computational methods for solving deterministic integration problems. By utilizing pseudorandom number generation, both the Hit-or-Miss (rejection) method and the Mean Value (sample average) method are implemented to evaluate integrals across single and multiple dimensions. The study spans fundamental geometric estimations, such as the computation of π , to complex multi-dimensional surface volumes bounded by non-linear constraints, including a paraboloid over a circular domain. A central focus is placed on the rigorous empirical verification of the algorithm's statistical error. Through logarithmic curve fitting of the absolute error against the number of iterations (N), the results successfully demonstrate that the integration error strictly adheres to the $O(N^{-0.5})$ convergence rate dictated by the Central Limit Theorem, completely independent of the integral's dimensionality.

Contents

1	Introduction	1
2	Numerical Implementation	1
2.1	Global Constants and Precision	1
2.2	The General Monte Carlo Solver	2
3	Area Under a Curve Using Monte Carlo Simulation	4
3.1	Problem 1: Area Under a Polynomial Curve	4
3.2	Problem 2: Area Under a Trigonometric Function	7
3.3	Problem 3: Area Between Two Curves	10
3.4	Problem 4: Area of a Quarter Circle	12
3.5	Problem 5: Monte Carlo Integration of a Difficult Function	14
4	Monte Carlo Simulation for Area Under a Surface	18
4.1	Problem 1: Area Under a Polynomial Curve	18
4.2	Problem 2: Parabolic Surface	20
4.3	Problem 3: Surface Over a Circular Region	23
4.4	Problem 4: Exponential Surface	25
4.5	Problem 5: Volume Between Two Surfaces	27
5	Remarks and Conclusion	29

1 Introduction

Numerical integration is a cornerstone of computational physics, essential for evaluating complex systems where analytical solutions are either intractable or impossible. While traditional deterministic algorithms, such as Simpson’s rule or Gaussian quadrature, are highly efficient for one-dimensional problems, they suffer severely from the ”curse of dimensionality.” As the number of variables increases, the computational cost of grid-based methods grows exponentially. Monte Carlo methods bypass this limitation by employing statistical randomness to approximate definite integrals, offering a convergence rate that remains invariant regardless of the system’s dimensions.

This report explores two distinct Monte Carlo architectures:

1. **The Hit-or-Miss Method:** A geometric approach that frames integration as a probability problem. By enclosing a target region within a known bounding volume and generating uniform random coordinates, the integral is estimated by the ratio of ”accepted” points that satisfy the bounding condition to the total number of samples.
2. **The Mean Value Method:** A statistical approach derived from the Mean Value Theorem for Definite Integrals. It approximates the integral by uniformly sampling the domain and calculating the expected (average) value of the function, scaled by the total volume of the integration space.

A defining characteristic of all Monte Carlo integration is its reliance on the Law of Large Numbers and the Central Limit Theorem. Unlike deterministic methods, the error in a Monte Carlo simulation is inherently statistical. Theory dictates that the standard deviation of the estimate and thus the absolute error decreases proportionally to $\frac{1}{\sqrt{N}}$, where N is the number of random samples. In this study, these theoretical frameworks are translated into highly optimized, vectorized computational algorithms. By evaluating a series of progressively complex geometries ranging from one-dimensional trigonometric functions to bounded three-dimensional surfaces, this report systematically visualizes the sampling behavior, tests the computational accuracy against exact analytical solutions, and rigorously proves the universal $N^{-0.5}$ error convergence rate.

2 Numerical Implementation

2.1 Global Constants and Precision

To ensure numerical stability and physical consistency across all problems, a shared module `constants_mod` is used. This module defines the double-precision floating-point kind (`real64`).

```

1 module constants_mod
2   implicit none
3   integer, parameter :: dp = selected_real_kind(15, 307)
4   real(dp), parameter :: pi = acos(-1.0_dp)
5 contains
6 end module constants_mod

```

Listing 1: Global Constants Module

2.2 The General Monte Carlo Solver

The core logic is encapsulated in the `monte_carlo_toolbox` module. This module consist of various functionality that is directly or indirectly solving the assignment problems. Here is the explanation:

1. **uniform:** This subroutine takes generates uniformly distributed random numbers in the domain $[a, b]$. It uses `random_number` subroutine which generates uniformly distributed random numbers between 0 and 1 and by mathematical transformation $x = a + (b - a)r$ it scales the existing random numbers (r) in our desired domain $[a, b]$.
2. **MCI_1D:** This function finds area made with x axis by a single curve using Monte Carlo mean-value method. It takes the integrand, lower bound, upper bound, number of random points and returns the area under the curve.
3. **MCI_2D:** This function finds volume made with xy plane by a surface (2D integrand) using Monte Carlo mean-value method. It takes the integrand, lower bound, upper bound (for both x and y axis), number of random points and returns the volume under the surface.
4. **MC_area_single:** This function finds area made with x axis by a single curve using Monte Carlo sampling method. It takes integrand, lower bound, upper bound, number of random points and returns the area under the curve.
5. **MC_area_dual:** This function finds area between two curve using Monte Carlo sampling method. It takes integrands, lower bound, upper bound, number of random points and returns the area between the curve.
6. **MC_volume_2D:** This function finds volume made with xy plane by a surface (2D integrand) using Monte Carlo sampling method. It takes integrand, lower bound, upper bound, number of random points, the boundary of points counting and returns the volume under the surface.

```

1 module monte_carlo_toolbox
2   use constants_mod
3   implicit none
4   abstract interface
5     function math_func_1D(x_val) result(y_val)
6       use constants_mod
7       real(dp), intent(in), dimension(:) :: x_val
8       real(dp), dimension(size(x_val)) :: y_val
9     end function math_func_1D
10
11     function math_func_2D(x_val, y_val) result(z_val)
12       use constants_mod
13       real(dp), intent(in), dimension(:) :: x_val
14       real(dp), intent(in), dimension(:) :: y_val
15       real(dp), dimension(size(x_val)) :: z_val
16     end function math_func_2D
17   end interface
18 contains
19   subroutine uniform(a, b, arr)
20     real(dp), intent(in) :: a, b

```

```

21      real(dp), intent(out), dimension(:) :: arr
22
23      call random_number(arr)
24      arr = a + (b - a)*arr
25  end subroutine uniform
26
27  function MCI_1D(f, a, b, N) result(A_mc)
28      procedure(math_func_1D) :: f
29      real(dp), intent(in) :: a, b
30      integer, intent(in) :: N
31      real(dp) :: A_mc
32      real(dp), allocatable :: x(:)
33
34      allocate(x(N))
35
36      call uniform(a, b, x)
37      A_mc = (b - a)*sum(f(x))/real(N, dp)
38      deallocate(x)
39  end function MCI_1D
40
41  function MCI_2D(f, x1, x2, y1, y2, N) result(A_mc)
42      procedure(math_func_2D) :: f
43      real(dp), intent(in) :: x1, x2, y1, y2
44      integer, intent(in) :: N
45      real(dp) :: A_mc
46      real(dp), allocatable :: x(:), y(:)
47
48      allocate(x(N), y(N))
49
50      call uniform(x1, x2, x)
51      call uniform(y1, y2, y)
52      A_mc = (x2 - x1)*(y2 - y1)*sum(f(x, y))/real(N, dp)
53      deallocate(x, y)
54  end function MCI_2D
55
56  function MC_area_single(f, N, a, b, c, d) result (A_mc)
57      procedure(math_func_1D) :: f
58      integer, intent(in) :: N
59      real(dp), intent(in) :: a, b, c, d
60      real(dp) :: A_mc
61
62      real(dp), allocatable :: x(:), y(:)
63      real(dp) :: area_box
64
65      allocate(x(N), y(N))
66      call uniform(a, b, x)
67      call uniform(c, d, y)
68
69      area_box = (b - a)*(d - c)
70      A_mc = area_box*real(count(y <= f(x)), dp)/real(N, dp)
71      deallocate(x, y)
72  end function MC_area_single
73
74  function MC_area_dual(f1, f2, N, a, b, c, d) result (A_mc)
75      procedure(math_func_1D) :: f1, f2
76      integer, intent(in) :: N

```

```

77     real(dp), intent(in) :: a, b, c, d
78     real(dp) :: A_mc
79
80     real(dp), allocatable :: x(:), y(:)
81     real(dp) :: area_box
82
83     allocate(x(N), y(N))
84     call uniform(a, b, x)
85     call uniform(c, d, y)
86
87     area_box = (b - a)*(d - c)
88     A_mc = area_box*real(count(y >= f1(x) .and. y <= f2(x)), dp)/real(
N, dp)
89     deallocate(x, y)
90 end function MC_area_dual
91
92 function MC_volume_2D(f, N, a, b, c, d, bound) result (A_mc)
93     procedure(math_func_2D) :: f
94     integer, intent(in) :: N
95     real(dp), intent(in) :: a, b, c, d, bound
96     real(dp) :: A_mc
97
98     real(dp), allocatable :: x(:), y(:)
99     real(dp) :: area_box
100
101     allocate(x(N), y(N))
102     call uniform(a, b, x)
103     call uniform(c, d, y)
104
105     area_box = (b - a)*(d - c)
106     A_mc = area_box*real(count( f(x,y) <= bound), dp)/real(N, dp)
107     deallocate(x, y)
108 end function MC_volume_2D
109 end module monte_carlo_toolbox

```

Listing 2: MC Toolbox Implementation

3 Area Under a Curve Using Monte Carlo Simulation

3.1 Problem 1: Area Under a Polynomial Curve

- **Function:** $f(x) = x^2$ in the interval $0 \leq x \leq 1$.
- **Code:** Monte carlo hit-or-miss method is used.

```

1 module prob01_physics
2     use constants_mod
3     use monte_carlo_toolbox
4     implicit none
5
6 contains
7     function quadratic_f(x_vals) result(y_vals)
8         real(dp), intent(in), dimension(:) :: x_vals
9         real(dp), dimension(size(x_vals)) :: y_vals
10

```

```

11      y_vals = x_vals**2.0_dp
12  end function quadratic_f
13
14  subroutine generate_scatter_data(N, a, b, filename)
15      integer, intent(in) :: N
16      character(len=*), intent(in) :: filename
17      real(dp), intent(in) :: a, b
18      real(dp), allocatable :: x(:), y(:), F1(:)
19      integer :: i, is_inside
20
21      allocate(x(N), y(N), F1(N))
22      call uniform(a, b, x)
23      call uniform(a, b, y)
24      F1 = quadratic_f(x)
25
26      open(unit=10, file=filename, status='replace')
27      write(10, *) "# X_coord   Y_coord   Inside_curve(1=Yes, 0=No)"
28
29      do i = 1, N
30          if (y(i) <= F1(i)) then
31              is_inside = 1
32          else
33              is_inside = 0
34          end if
35          write(10, '(2F10.5, I5)') x(i), y(i), is_inside
36      end do
37
38      close(10)
39      deallocate(x, y, F1)
40  end subroutine generate_scatter_data
41
42  subroutine generate_error_data(th_val, a, b, filename)
43      real(dp), intent(in) :: th_val
44      character(len=*), intent(in) :: filename
45      real(dp), intent(in) :: a, b
46      integer :: i, N
47      real(dp) :: current_val
48
49      open(unit=20, file=filename, status='replace')
50      write(20, *) "# N_iterations   Estimated_values"
51
52      do i = 2, 5
53          N = 10**i
54          current_val = MC_area_single(quadratic_f, N, a, b, a, b)
55          write(20, '(I10, F15.9)') N, abs(current_val - th_val)
56      end do
57      close(20)
58  end subroutine generate_error_data
59 end module prob01_physics
60
61 program prob01_driver
62     use prob01_physics
63     use constants_mod
64     implicit none
65     integer :: N

```

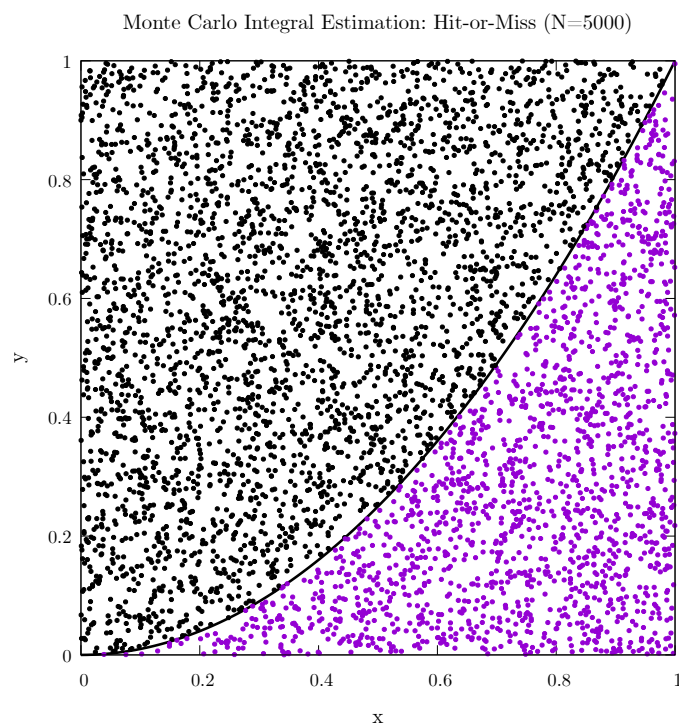


```

66  real(dp) :: a, b, integral, exact
67
68  a = 0
69  b = 1
70  N = 5000
71  exact = 1.0_dp/3.0_dp
72
73  call random_seed()
74  integral = MC_area_single(quadratic_f, N, a, b, a, b)
75
76  write(*, '(A)') "Function: f(x) = x^2"
77  write(*, '(A)') "-----"
78  write(*, '(A, G0.6)') "Monte-Carlo estimation: ", integral
79  write(*, '(A, G0.6)') "Exact result: ", exact
80
81  call generate_scatter_data(N, a, b, 'data/prob01_sampling.dat')
82  call generate_error_data(exact, a, b, 'data/prob01_error.dat')
83
84  end program prob01_driver

```

- **Sampling Plot:** The function $f(x) = x^2$ is plotted along with the randomly generated points inside the 1×1 rectangular grid. The 'Black' points reside outside the curve and the 'Violet' points reside inside the curve traced by the function.



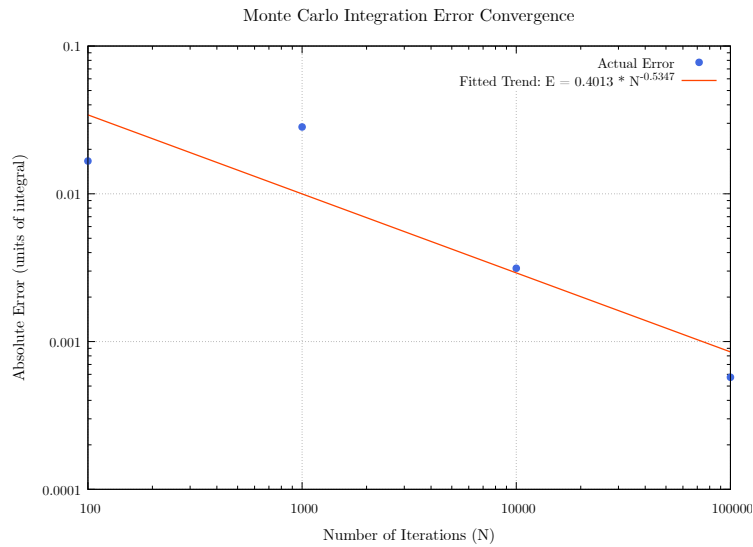
- **Output:** Comparison with the exact value.

```

Function: f(x) = x^2
-----
Monte-Carlo estimation: 0.334000
Exact result: 0.333333

```

- **Convergence Plot:**



The log log plot of error convergence against the number of random points (N) with a slope ≈ 0.5 successfully shows that the error propagation in Monte Carlo method is proportional to $\frac{1}{\sqrt{N}}$ where, N is the number of points injected inside the rectangular grid.

3.2 Problem 2: Area Under a Trigonometric Function

- **Function:** $f(x) = \sin x$ for $0 \leq x \leq \pi$. For this problem, the bounding rectangle will have length of π unit ($0 \leq x \leq \pi$) and breadth of 1 unit ($0 \leq y \leq 1$) as $\sin x$ has upper bound of 1.
- **Code:** Monte Carlo sampling method is used.

```

1 module prob02_physics
2   use constants_mod
3   use monte_carlo_toolbox
4   implicit none
5
6 contains
7   function trig_f(x_vals) result(y_vals)
8     real(dp), intent(in), dimension(:) :: x_vals
9     real(dp), dimension(size(x_vals)) :: y_vals
10
11     y_vals = sin(x_vals)
12   end function trig_f
13
14   subroutine generate_scatter_data(N, a, b, c, d, filename)
15     integer, intent(in) :: N
16     character(len=*), intent(in) :: filename
17     real(dp), intent(in) :: a, b, c, d
18     real(dp), allocatable :: x(:), y(:), F1(:)
19     integer :: i, is_inside
20
21     allocate(x(N), y(N), F1(N))

```

```

22     call uniform(a, b, x)
23     call uniform(c, d, y)
24     F1 = trig_f(x)
25
26     open(unit=10, file=filename, status='replace')
27     write(10, *) "# X_coord   Y_coord   Inside_curve(1=Yes, 0=No)
"
28
29     do i = 1, N
30         if (y(i) <= F1(i)) then
31             is_inside = 1
32         else
33             is_inside = 0
34         end if
35         write(10, '(2F10.5, I5)') x(i), y(i), is_inside
36     end do
37
38     close(10)
39     deallocate(x, y, F1)
40 end subroutine generate_scatter_data
41
42 subroutine generate_error_data(th_val, a, b, c, d, filename)
43     real(dp), intent(in) :: th_val
44     character(len=*), intent(in) :: filename
45     real(dp), intent(in) :: a, b, c, d
46     integer :: i, N
47     real(dp) :: current_val
48
49     open(unit=20, file=filename, status='replace')
50     write(20, *) "# N_iterations   Estimated_values"
51
52     do i = 1, 7
53         N = 10**i
54         current_val = MC_area_single(trig_f, N, a, b, c, d)
55         write(20, '(I10, F15.9)') N, abs(current_val - th_val)
56     end do
57     close(20)
58 end subroutine generate_error_data
59 end module prob02_physics
60
61 program prob02_driver
62     use prob02_physics
63     use constants_mod
64     implicit none
65     integer :: N
66     real(dp) :: a, b, c, d, integral, exact
67
68     a = 0
69     b = pi
70     c = 0
71     d = 1
72     N = 5000
73     exact = 2.0_dp
74
75     call random_seed()
76     integral = MC_area_single(trig_f, N, a, b, c, d)

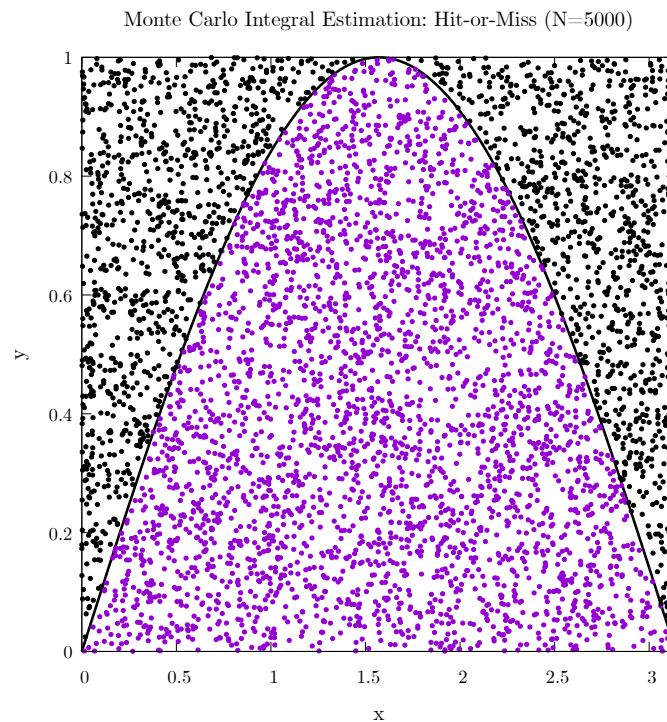
```

```

77
78     write(*, '(A)') "Function: f(x) = sin(x)"
79     write(*, '(A)') "-----"
80     write(*, '(A, G0.6)') "Monte-Carlo estimation: ", integral
81     write(*, '(A, G0.6)') "Exact result: ", exact
82
83     call generate_scatter_data(N, a, b, c, d, 'data/prob02_sampling.
84     dat')
85     call generate_error_data(exact, a, b, c, d, 'data/prob02_error.
86     dat')
87
88 end program prob02_driver

```

- **Sampling Plot:** The function $f(x) = \sin x$ is plotted along with the randomly generated points inside the $\pi \times 1$ rectangular grid. The 'Black' points reside outside the curve and the 'Violet' points reside inside the curve traced by the function.



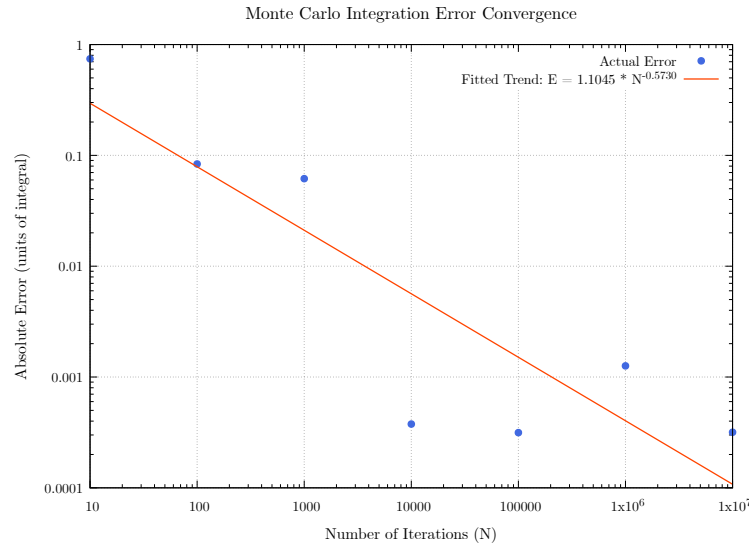
- **Output:**

```

Function: f(x) = sin(x)
-----
Monte-Carlo estimation: 1.97292
Exact result: 2.00000

```

- **Convergence Plot:**



The log log plot of error convergence against the number of random points (N) with a slope ≈ 0.5 successfully shows that the error propagation in Monte Carlo method is proportional to $\frac{1}{\sqrt{N}}$ where, N is the number of points injected inside the rectangular grid.

3.3 Problem 3: Area Between Two Curves

- **Functions:** $y_1 = x^2$ and $y_2 = x$ for $0 \leq x \leq 1$. For this problem, the bounding rectangle will be a square of length of 1 unit ($0 \leq x \leq 1$) and breadth of 1 unit ($0 \leq y \leq 1$) as both the functions intersect at $(1, 1)$.
- **Code:**

```

1 module prob03_physics
2     use constants_mod
3     use monte_carlo_toolbox
4     implicit none
5 contains
6     function quadratic_f(x_vals) result(y_vals)
7         real(dp), intent(in), dimension(:) :: x_vals
8         real(dp), dimension(size(x_vals)) :: y_vals
9
10        y_vals = x_vals**2.0_dp
11    end function quadratic_f
12
13    function linear_f(x_vals) result(y_vals)
14        real(dp), intent(in), dimension(:) :: x_vals
15        real(dp), dimension(size(x_vals)) :: y_vals
16
17        y_vals = x_vals
18    end function linear_f
19
20    subroutine generate_scatter_data(N, a, b, filename)
21        integer, intent(in) :: N
22        character(len=*), intent(in) :: filename
23        real(dp), intent(in) :: a, b

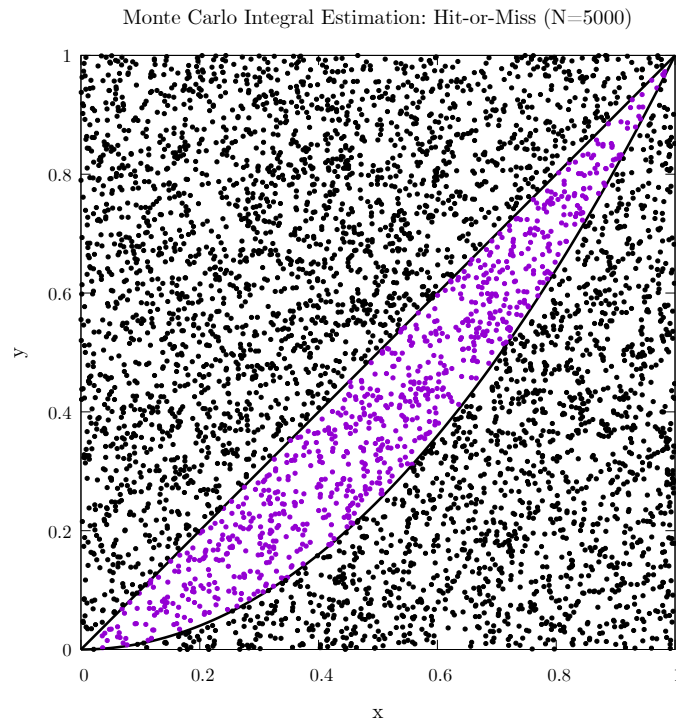
```

```

24     real(dp), allocatable :: x(:), y(:), F1(:), F2(:)
25     integer :: i, is_inside
26
27     allocate(x(N), y(N), F1(N), F2(N))
28     call uniform(a, b, x)
29     call uniform(a, b, y)
30     F1 = quadratic_f(x)
31     F2 = linear_f(x)
32
33     open(unit=10, file=filename, status='replace')
34     write(10, *) "# X_coord   Y_coord   Inside_curve(1=Yes, 0=No)
35
36     do i = 1, N
37         if (y(i) >= F1(i) .and. y(i) <= F2(i)) then
38             is_inside = 1
39         else
40             is_inside = 0
41         end if
42         write(10, '(2F10.5, I5)') x(i), y(i), is_inside
43     end do
44
45     close(10)
46     deallocate(x, y, F1, F2)
47 end subroutine generate_scatter_data
48 end module prob03_physics
49
50 program prob03_driver
51     use prob03_physics
52     use constants_mod
53     use monte_carlo_toolbox
54     implicit none
55     integer :: N
56     real(dp) :: a, b, integral, exact
57
58     a = 0
59     b = 1
60     N = 5000
61     exact = 1.0_dp/6.0_dp
62
63     call random_seed()
64     integral = MC_area_dual(quadratic_f, linear_f, N, a, b, a, b)
65
66     write(*, '(A)') "Function: f1(x) = x^2 and f2(x) = x"
67     write(*, '(A)') "-----"
68     write(*, '(A, G0.6)') "Monte-Carlo estimation: ", integral
69     write(*, '(A, G0.6)') "Exact result: ", exact
70
71     call generate_scatter_data(N, a, b, 'data/prob03_sampling.dat')
72
73 end program prob03_driver

```

- **Sampling Plot:** The functions $y_1 = x^2$ and $y_2 = x$ are plotted along with the randomly generated points inside the 1×1 rectangular grid. The 'Black' points reside outside the bounded region and the 'Violet' points reside inside the bounded region traced by the functions.



- **Output:**

```
Function: f1(x) = x^2 and f2(x) = x
```

```
-----  
Monte-Carlo estimation: 0.173800
```

```
Exact result: 0.166667
```

3.4 Problem 4: Area of a Quarter Circle

- **Function:** $y = \sqrt{1 - x^2}$ for $0 \leq x \leq 1$.

- **Code:**

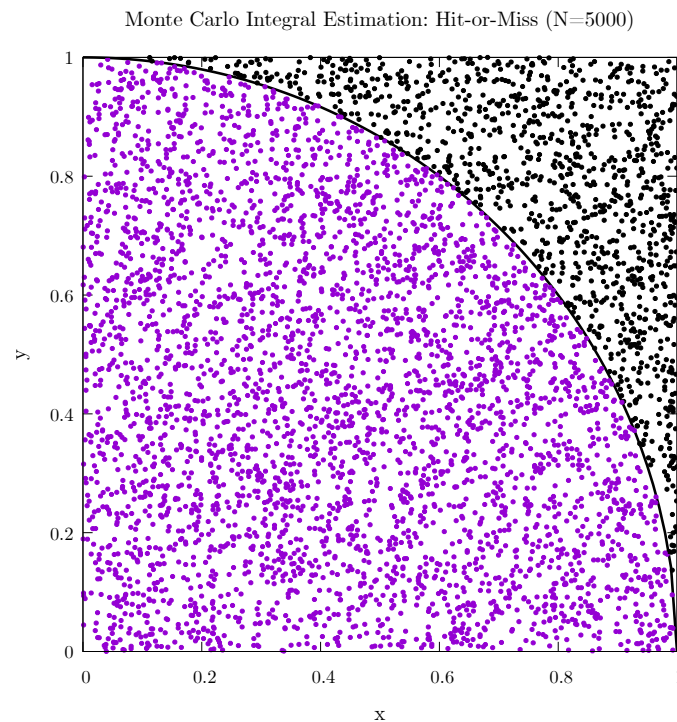
```
1 module prob04_physics
2     use constants_mod
3     use monte_carlo_toolbox
4     implicit none
5 contains
6     function quarter_f(x_vals) result(y_vals)
7         real(dp), intent(in), dimension(:) :: x_vals
8         real(dp), dimension(size(x_vals)) :: y_vals
9
10        y_vals = sqrt(1 - x_vals**2.0_dp)
11    end function quarter_f
12
13    subroutine generate_scatter_data(N, a, b, filename)
14        integer, intent(in) :: N
15        character(len=*), intent(in) :: filename
16        real(dp), intent(in) :: a, b
17        real(dp), allocatable :: x(:), y(:), F1(:)
```

```

18     integer :: i, is_inside
19
20     allocate(x(N), y(N), F1(N))
21     call uniform(a, b, x)
22     call uniform(a, b, y)
23     F1 = quarter_f(x)
24
25     open(unit=10, file=filename, status='replace')
26     write(10, *) "# X_coord   Y_coord   Inside_curve(1=Yes, 0=No)
27
28
29     do i = 1, N
30         if (y(i) <= F1(i)) then
31             is_inside = 1
32         else
33             is_inside = 0
34         end if
35         write(10, '(2F10.5, I5)') x(i), y(i), is_inside
36     end do
37
38     close(10)
39     deallocate(x, y, F1)
40 end subroutine generate_scatter_data
41
42 program prob04_driver
43     use prob04_physics
44     use constants_mod
45     use monte_carlo_toolbox
46     implicit none
47     integer :: N
48     real(dp) :: a, b, integral
49
50     a = 0.0_dp
51     b = 1.0_dp
52     N = 5000
53
54     call random_seed()
55     integral = MC_area_single(quarter_f, N, a, b, a, b)
56
57     write(*, '(A)') "Function: f(x) = (1 - x^2)^0.5"
58     write(*, '(A)') "-----"
59     write(*, '(A, G0.6)') "Monte-Carlo estimation: ", integral
60     write(*, '(A, G0.10)') "Estimated value of pi: ", 4.0_dp*integral
61
62     call generate_scatter_data(N, a, b, 'data/prob04_sampling.dat')
63
64 end program prob04_driver

```

- **Sampling Plot:** The function $y = \sqrt{1-x^2}$ is plotted along with the randomly generated points inside the 1×1 rectangular grid. The 'Black' points reside outside the curve and the 'Violet' points reside inside the curve traced by the quarter circle.



- **Output:**

```
Function: f(x) = (1 - x^2)^0.5
-----
Monte-Carlo estimation: 0.786600
Estimated value of pi: 3.146400000
```

3.5 Problem 5: Monte Carlo Integration of a Difficult Function

- **Function:** $y = e^{-x^2}$ for $0 \leq x \leq 2$.
- **Code:** Monte Carlo mean-value method is used. The key formula is

$$A_{MC} = (b - a) \frac{1}{N} \sum_{i=1}^N f(x_i)$$

where x_i are the uniform random numbers inside the desired integration domain.

```
1 module prob05_physics
2     use monte_carlo_toolbox
3     use constants_mod
4     implicit none
5 contains
6     function f_exp(x_val) result(y_val)
7         real(dp), intent(in), dimension(:) :: x_val
8         real(dp), dimension(size(x_val)) :: y_val
9
10        y_val = exp(-x_val**2)
11    end function f_exp
```

```

12
13  subroutine generate_func_data(f, a, b, N, filename)
14      procedure(math_func_1D) :: f
15      real(dp), intent(in) :: a, b
16      integer, intent(in) :: N
17      character(len=*), intent(in) :: filename
18      real(dp), allocatable :: x(:), y(:)
19      integer :: i
20
21      allocate(x(N), y(N))
22      call uniform(a, b, x)
23      y = f(x)
24      open(unit=10, file=filename, status='replace')
25      write(10, *) "# x    f(x)"
26
27      do i = 1, N
28          write(10, '(2F10.5)') x(i), y(i)
29      end do
30
31      close(10)
32      deallocate(x, y)
33  end subroutine generate_func_data
34
35  subroutine generate_convergence_data(a, b, filename)
36      real(dp), intent(in) :: a, b
37      character(len=*), intent(in) :: filename
38      integer :: i, N
39      real(dp) :: current_val
40
41      open(unit=20, file=filename, status='replace')
42      write(20, *) "# N_iterations    Estimated_values"
43
44      do i = 1, 7
45          N = 10**i
46          current_val = MCI_1D(f_exp, a, b, N)
47          write(20, '(I10, F15.9)') N, current_val
48      end do
49
50      close(20)
51  end subroutine generate_convergence_data
52  end module prob05_physics
53
54
55  program prob05_driver
56      use constants_mod
57      use monte_carlo_toolbox
58      use prob05_physics
59      implicit none
60      real(dp) :: a, b, I1, I2, I3
61      integer :: N
62
63      a = 0.0_dp
64      b = 2.0_dp
65      N = 10000
66
67      call random_seed()

```

```

68     I1 = MCI_1D(f_exp, a, b, 1000)
69     I2 = MCI_1D(f_exp, a, b, 10000)
70     I3 = MCI_1D(f_exp, a, b, 100000)
71
72     write(*, '(A)') "Function: f(x) = e^(-x^2)"
73     write(*, '(A)') "-----"
74
75     write(*, '(A)') "Estimated values: "
76     write(*, '(A)') "-----+-----+-----"
77
78     write(*, '(A)') "N = 1000      | N = 10000      | N = 100000"
79     write(*, '(A)') "-----+-----+-----"
80
81     write(*, '(G0.10, A, G0.10, A, G0.10)') I1, " | ", I2, " | ", I3
82     write(*, '(A)') "-----+-----+-----"
83
84     call generate_func_data(f_exp, a, b, 200, 'data/prob05_sampling.
85     dat')
86     call generate_convergence_data(a, b, 'data/prob05_value_conv.dat'
87     )
88
89 end program prob05_driver

```

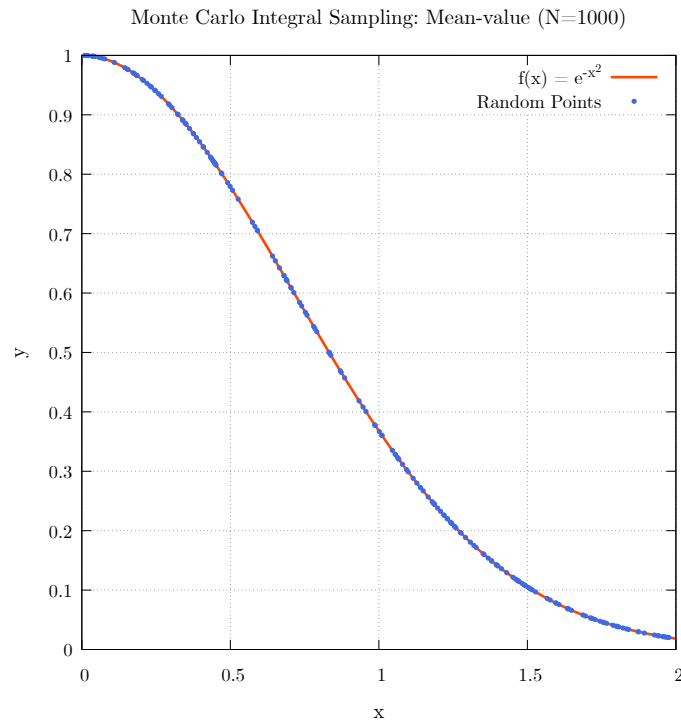
• Output:

```

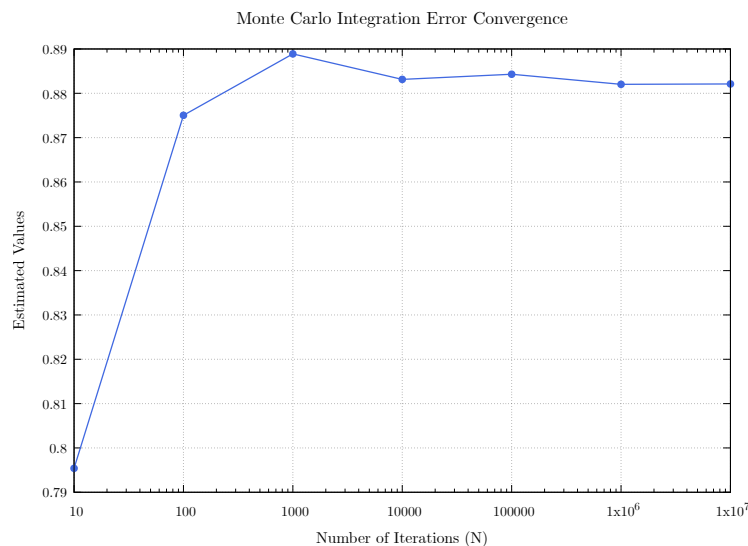
Function: f(x) = e^(-x^2)
-----
Estimated values:
-----+-----+-----
N = 1000      | N = 10000      | N = 100000
-----+-----+-----
0.8861357146 | 0.8846096080  | 0.8825903875
-----+-----+-----

```

- **Sampling Plot:** The functional value is calculated for the randomly generated points x_i and plotted along with the function $y = e^{-x^2}$ inside the 2×1 rectangular grid. We expect the points will be on the curve with a random distribution.



- **Convergence Plot:** The log plot of value convergence shows that as we increase the number of random points (N), the value of the integral is converging towards a specific value. For the finite interval $[0, 2]$, the value is close to the infinite limit but not exactly equal. The exact value is $\frac{\sqrt{\pi}}{2}\text{erf}(2)$.



Monte Carlo Integration is highly useful for complex, non-analytical or high-dimensional integrals because it approximates results through random sampling rather than analytical calculation. By averaging function evaluations at random points, it provides an efficient, dimension-independent convergence rate $\left(\mathcal{O}\left(\frac{1}{\sqrt{N}}\right)\right)$, making it superior for problems with

high dimensions, irregular shapes, or discontinuities where traditional grid-based methods fail.

4 Monte Carlo Simulation for Area Under a Surface

4.1 Problem 1: Area Under a Polynomial Curve

- **Function:** $z = x + y$ in the interval $0 \leq x \leq 1$ and $0 \leq y \leq 1$.

- **Code:**

```

1 module prob01_physics
2     use constants_mod
3     use monte_carlo_toolbox
4     implicit none
5
6 contains
7     function quadratic_f(x_vals) result(y_vals)
8         real(dp), intent(in), dimension(:) :: x_vals
9         real(dp), dimension(size(x_vals)) :: y_vals
10
11         y_vals = x_vals**2.0_dp
12     end function quadratic_f
13
14     subroutine generate_scatter_data(N, a, b, filename)
15         integer, intent(in) :: N
16         character(len=*), intent(in) :: filename
17         real(dp), intent(in) :: a, b
18         real(dp), allocatable :: x(:), y(:), F1(:)
19         integer :: i, is_inside
20
21         allocate(x(N), y(N), F1(N))
22         call uniform(a, b, x)
23         call uniform(a, b, y)
24         F1 = quadratic_f(x)
25
26         open(unit=10, file=filename, status='replace')
27         write(10, *) "# X_coord   Y_coord   Inside_curve(1=Yes, 0=No)"
28
29         do i = 1, N
30             if (y(i) <= F1(i)) then
31                 is_inside = 1
32             else
33                 is_inside = 0
34             end if
35             write(10, '(2F10.5, I5)') x(i), y(i), is_inside
36         end do
37
38         close(10)
39         deallocate(x, y, F1)
40     end subroutine generate_scatter_data
41
42     subroutine generate_error_data(th_val, a, b, filename)
43         real(dp), intent(in) :: th_val

```

```

44     character(len=*), intent(in) :: filename
45     real(dp), intent(in) :: a, b
46     integer :: i, N
47     real(dp) :: current_val
48
49     open(unit=20, file=filename, status='replace')
50     write(20, *) "# N_iterations    Estimated_values"
51
52     do i = 2, 5
53         N = 10**i
54         current_val = MC_area_single(quadratic_f, N, a, b, a, b)
55         write(20, '(I10, F15.9)') N, abs(current_val - th_val)
56     end do
57     close(20)
58 end subroutine generate_error_data
59 end module prob01_physics
60
61 program prob01_driver
62     use prob01_physics
63     use constants_mod
64     implicit none
65     integer :: N
66     real(dp) :: a, b, integral, exact
67
68     a = 0
69     b = 1
70     N = 5000
71     exact = 1.0_dp/3.0_dp
72
73     call random_seed()
74     integral = MC_area_single(quadratic_f, N, a, b, a, b)
75
76     write(*, '(A)') "Function: f(x) = x^2"
77     write(*, '(A)') "-----"
78     write(*, '(A, G0.6)') "Monte-Carlo estimation: ", integral
79     write(*, '(A, G0.6)') "Exact result: ", exact
80
81     call generate_scatter_data(N, a, b, 'data/prob01_sampling.dat')
82     call generate_error_data(exact, a, b, 'data/prob01_error.dat')
83
84 end program prob01_driver

```

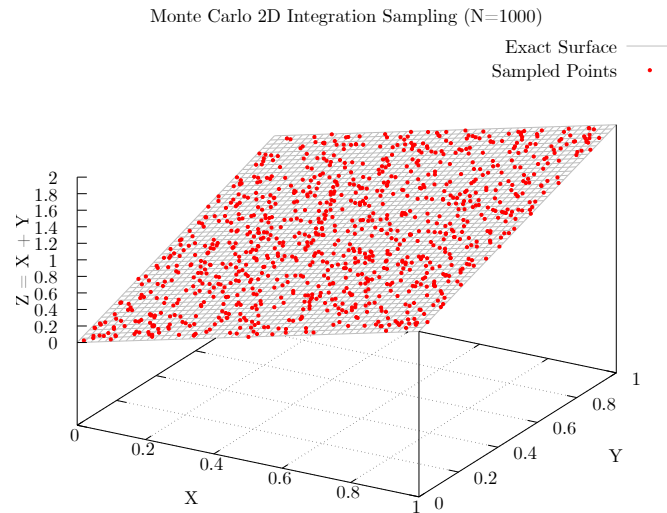
- **Output:** Comparison with the exact value.

```

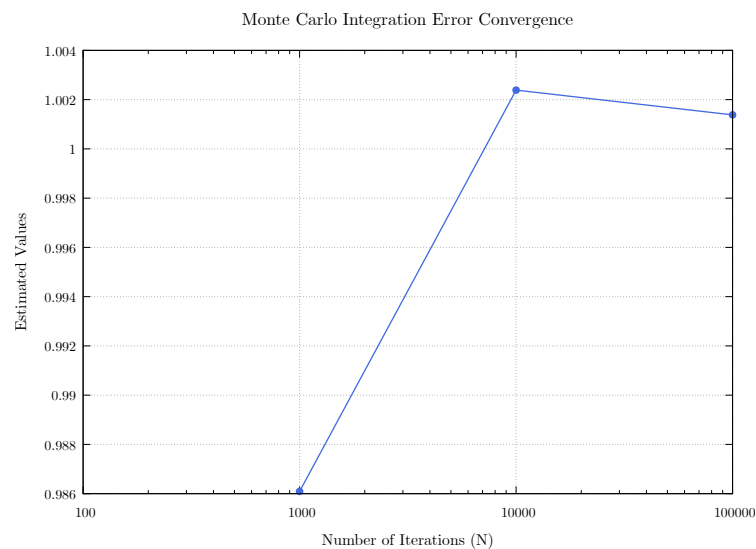
Function: f(x) = x + y
-----
Estimated value: 0.9994654945
Actual value: 1.0000000000

```

- **Sampling Plot:** The function $f(x, y) = x + y$ is plotted along with the randomly generated points (x_i, y_i) inside the 1×1 square grid using 3D plot. We expect the points will be on the plotted surface with a random distribution.



- **Convergence Plot:** The log plot of value convergence shows that as we increase the number of random points (N), the value of the integral is converging towards the analytical value "1".



4.2 Problem 2: Parabolic Surface

- **Function:** $z = x^2 + y^2$ in the interval $0 \leq x \leq 1$ and $0 \leq y \leq 1$.
- **Code:**

```

1 module prob07_physics
2     use constants_mod
3     use monte_carlo_toolbox
4     implicit none
5 contains
6     function paraboloid_f(x_val, y_val) result(z_val)

```

```

7      real(dp), intent(in), dimension(:) :: x_val, y_val
8      real(dp), dimension(size(x_val)) :: z_val
9
10     z_val = x_val**2 + y_val**2
11 end function paraboloid_f
12
13 subroutine generate_sampling_data(f, x1, x2, y1, y2, N, filename)
14     procedure(math_func_2D) :: f
15     integer, intent(in) :: N
16     character(len=*), intent(in) :: filename
17     real(dp), intent(in) :: x1, x2, y1, y2
18     real(dp), allocatable :: x(:), y(:), z(:)
19     integer :: i
20
21     allocate(x(N), y(N), z(N))
22     call uniform(x1, x2, x)
23     call uniform(y1, y2, y)
24     z = f(x, y)
25     open(unit=10, file=filename, status='replace')
26     write(10, *) "# x          y          f(x, y)"
27
28     do i = 1, N
29         write(10, '(3F10.5)') x(i), y(i), z(i)
30     end do
31
32     close(10)
33     deallocate(x, y, z)
34 end subroutine generate_sampling_data
35
36 subroutine generate_convergence_data(th_val, x1, x2, y1, y2,
filename)
37     real(dp), intent(in) :: th_val
38     character(len=*), intent(in) :: filename
39     real(dp), intent(in) :: x1, x2, y1, y2
40     integer :: i, N
41     real(dp) :: current_val
42
43     open(unit=20, file=filename, status='replace')
44     write(20, *) "# N_iterations    Estimated_values"
45
46     do i = 1, 7
47         N = 10**i
48         current_val = MCI_2D(paraboloid_f, x1, x2, y1, y2, N)
49         write(20, '(I10, F15.9)') N, abs(current_val - th_val)
50     end do
51
52     close(20)
53 end subroutine generate_convergence_data
54 end module prob07_physics
55
56
57 program prob07_driver
58     use constants_mod
59     use monte_carlo_toolbox
60     use prob07_physics
61     implicit none

```

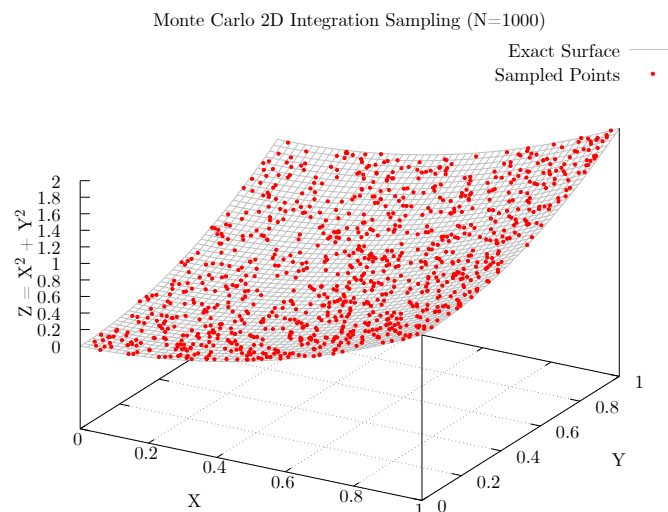


```

62  real(dp) :: x1, x2, y1, y2, exact, integral
63  integer :: N
64
65  N = 10000
66
67  x1 = 0.0_dp
68  x2 = 1.0_dp
69  y1 = 0.0_dp
70  y2 = 1.0_dp
71  exact = 2.0_dp/3.0_dp
72
73  call random_seed()
74  integral = MCI_2D(paraboloid_f, x1, x2, y1, y2, N)
75
76  write(*, '(A)') "Function: f(x) = x^2 + y^2"
77  write(*, '(A)') "-----"
78  write(*, '(A, G0.10)') "Estimated value: ", integral
79  write(*, '(A, G0.10)') "Actual value: ", exact
80
81  call generate_sampling_data(paraboloid_f, x1, x2, y1, y2, 1000, '
data/prob07_sampling.dat')
82  call generate_convergence_data(exact, x1, x2, y1, y2, 'data/
prob07_error.dat')
83
84  end program prob07_driver

```

- **Sampling Plot:** The function $f(x, y) = x^2 + y^2$ is plotted along with the randomly generated points (x_i, y_i) inside the 1×1 square grid using 3D plot. We expect the points will be on the plotted surface with a random distribution.



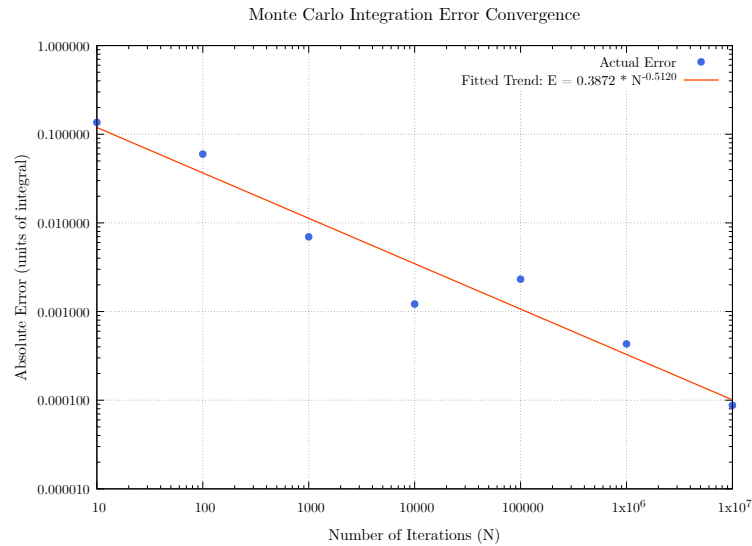
- **Output:** Comparison with the exact value.

```
Function: f(x) = x^2 + y^2
```

```
-----
```

Estimated value: 0.6527729435
 Actual value: 0.6666666667

- **Convergence Plot:**



The log log plot of error convergence against the number of random points (N) with a slope ≈ 0.5 successfully shows that the error propagation in Monte Carlo method is proportional to $\frac{1}{\sqrt{N}}$ where, N is the number of points injected inside the square grid.

4.3 Problem 3: Surface Over a Circular Region

- **Function:** $z = x^2 + y^2$ in the interval $0 \leq x \leq 1$ inside the unit circle $x^2 + y^2 \leq 1$.

- **Code:**

```

1 module prob08_physics
2   use constants_mod
3   use monte_carlo_toolbox
4   implicit none
5
6 contains
7   function unit_circle(x_val, y_val) result(z_val)
8     use constants_mod
9     real(dp), intent(in), dimension(:) :: x_val, y_val
10    real(dp), dimension(size(x_val)) :: z_val
11
12    z_val = x_val**2 + y_val**2
13  end function unit_circle
14
15  subroutine generate_scatter_data(N, a, b, bound, filename)
16    integer, intent(in) :: N
17    character(len=*), intent(in) :: filename
18    real(dp), intent(in) :: a, b, bound
19    real(dp), allocatable :: x(:), y(:), F1(:)
20    integer :: i, is_inside

```

```

21
22     allocate(x(N), y(N), F1(N))
23     call uniform(a, b, x)
24     call uniform(a, b, y)
25     F1 = unit_circle(x, y)
26
27     open(unit=10, file=filename, status='replace')
28     write(10, *) "# X_coord   Y_coord   Inside_curve(1=Yes, 0=No)
29
30     do i = 1, N
31         if (F1(i) <= bound) then
32             is_inside = 1
33         else
34             is_inside = 0
35         end if
36         write(10, '(2F10.5, I5, F10.5)') x(i), y(i), is_inside,
37         F1(i)
38     end do
39
40     close(10)
41     deallocate(x, y, F1)
42 end subroutine generate_scatter_data
43
44 program prob08_driver
45     use constants_mod
46     use monte_carlo_toolbox
47     use prob08_physics
48     implicit none
49     real(dp) :: exact, bound, a, b, integral
50     integer :: N
51
52     N = 10000
53     a = -1.0_dp
54     b = 1.0_dp
55     bound = 1.0_dp
56     exact = pi
57
58     call random_seed()
59     integral = MC_volume_2D(unit_circle, N, a, b, a, b, bound)
60
61     write(*, '(A, G0.6)') "Monte-Carlo estimation: ", integral
62     write(*, '(A, G0.6)') "Exact result: ", exact
63
64     call generate_scatter_data(2000, a, b, bound, 'data/
65     prob08_sampling.dat')
66 end program prob08_driver

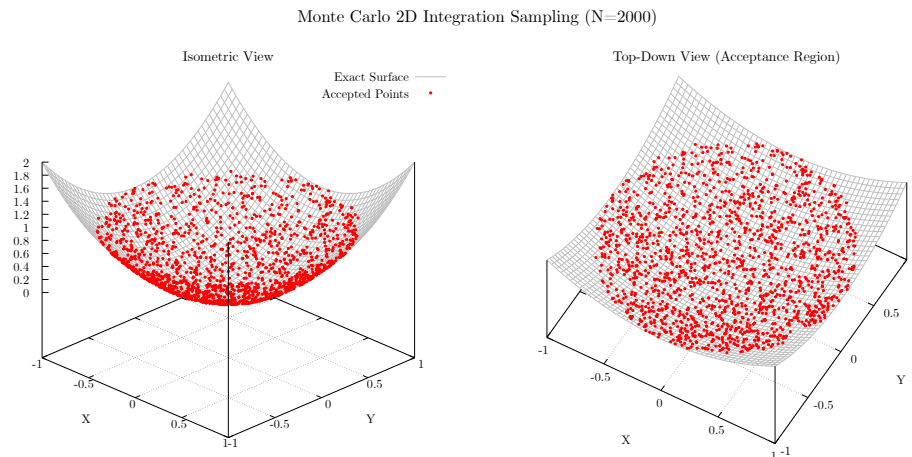
```

- **Output:**

<pre> Monte-Carlo estimation: 3.12120 Exact result: 3.14159 </pre>
--

- **Sampling Plot:** The randomly generated points are plotted in a 1×1 square grid.

The 'Violet' points satisfies the condition $x^2 + y^2 \leq 1$ the 'Black' points violates this condition.



4.4 Problem 4: Exponential Surface

- **Function:** $z = e^{-(x^2+y^2)}$ for $-2 \leq x \leq 2$ and $-2 \leq y \leq 2$.

- **Code:**

```

1 module prob09_physics
2   use constants_mod
3   use monte_carlo_toolbox
4   implicit none
5 contains
6   function gaussian_2D(x_val, y_val) result(z_val)
7     real(dp), intent(in), dimension(:) :: x_val, y_val
8     real(dp), dimension(size(x_val)) :: z_val
9
10    z_val = exp(-(x_val**2 + y_val**2))
11  end function gaussian_2D
12
13  subroutine generate_sampling_data(f, x1, x2, y1, y2, N, filename)
14    procedure(math_func_2D) :: f
15    integer, intent(in) :: N
16    character(len=*), intent(in) :: filename
17    real(dp), intent(in) :: x1, x2, y1, y2
18    real(dp), allocatable :: x(:), y(:), z(:)
19    integer :: i
20
21    allocate(x(N), y(N), z(N))
22    call uniform(x1, x2, x)
23    call uniform(y1, y2, y)
24    z = f(x, y)
25    open(unit=10, file=filename, status='replace')
26    write(10, *) "# x      y      f(x, y)"
27
28    do i = 1, N
29      write(10, '(3F10.5)') x(i), y(i), z(i)
30    end do

```

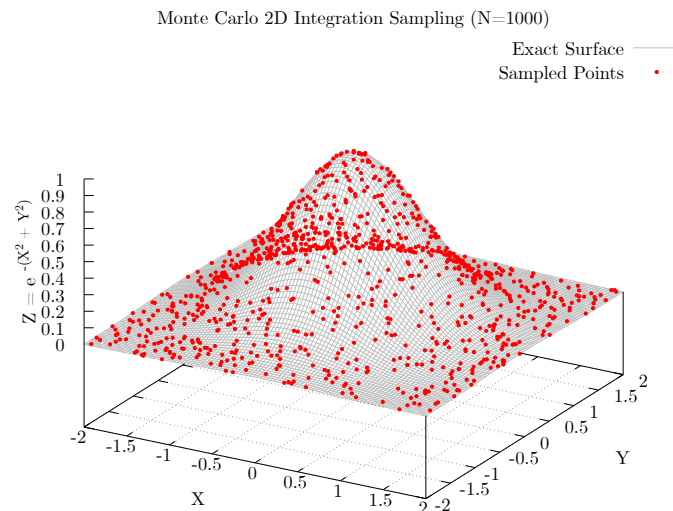
```

31
32     close(10)
33     deallocate(x, y, z)
34 end subroutine generate_sampling_data
35
36 subroutine generate_convergence_data(x1, x2, y1, y2, filename)
37     character(len=*) , intent(in) :: filename
38     real(dp) , intent(in) :: x1, x2, y1, y2
39     integer :: i, N
40     real(dp) :: current_val
41
42     open(unit=20, file=filename, status='replace')
43     write(20, *) "# N_iterations    Estimated_values"
44
45     do i = 1, 7
46         N = 10**i
47         current_val = MCI_2D(gaussian_2D, x1, x2, y1, y2, N)
48         write(20, '(I10, F15.9)') N, current_val
49     end do
50
51     close(20)
52 end subroutine generate_convergence_data
53 end module prob09_physics
54
55
56 program prob09_driver
57     use constants_mod
58     use monte_carlo_toolbox
59     use prob09_physics
60     implicit none
61     real(dp) :: x1, x2, y1, y2, integral
62     integer :: N
63
64     N = 10000
65
66     x1 = -2.0_dp
67     x2 = 2.0_dp
68     y1 = -2.0_dp
69     y2 = 2.0_dp
70
71     call random_seed()
72     integral = MCI_2D(gaussian_2D, x1, x2, y1, y2, N)
73
74     write(*, '(A)') "Function: f(x) = e^(-(x^2+y^2))"
75     write(*, '(A)') "-----"
76     write(*, '(A, G0.10)') "Estimated value: ", integral
77
78     call generate_sampling_data(gaussian_2D, x1, x2, y1, y2, 1000, '
data/prob09_sampling.dat')
79     call generate_convergence_data(x1, x2, y1, y2, 'data/
prob09_value_conv.dat')
80
81 end program prob09_driver

```

- **Sampling Plot:** The function $f(x, y) = x^2 + y^2$ is plotted along with the randomly generated points (x_i, y_i) inside the 4×4 square grid using 3D plot. We expect the

points will be on the plotted surface with a random distribution.

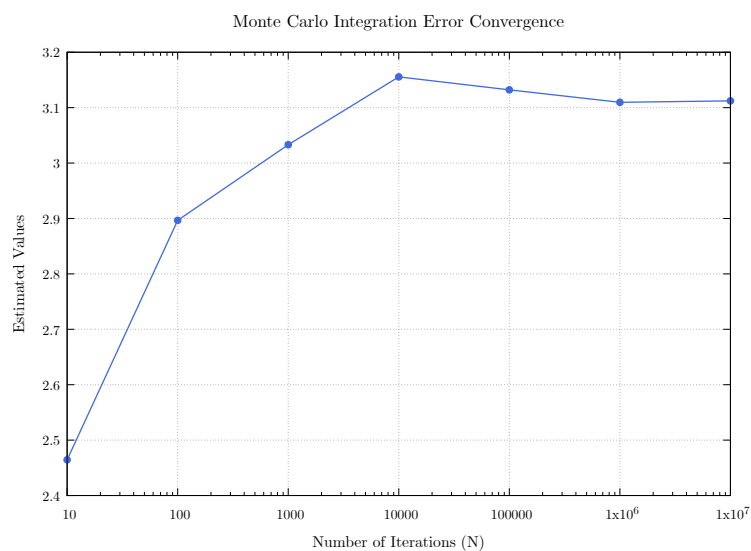


- **Output:**

Function: $f(x) = e^{-(x^2 + y^2)}$

Estimated value: 3.071421466

- **Convergence Plot:** The log plot of value convergence shows that as we increase the number of random points (N), the value of the integral is converging towards a specific value (around π solved using polar coordinates.)



4.5 Problem 5: Volume Between Two Surfaces

- **Function:** $z_1 = x + y$ and $z_2 = x^2 + y^2$ for $0 \leq x \leq 1$ and $0 \leq y \leq 1$.

• Code:

```

1 module prob10_physics
2   use constants_mod
3   use monte_carlo_toolbox
4   implicit none
5 contains
6   function integrand(x_val, y_val) result(z_val)
7     real(dp), intent(in), dimension(:) :: x_val, y_val
8     real(dp), dimension(size(x_val)) :: z_val
9
10    z_val = (x_val + y_val) - (x_val**2 + y_val**2)
11  end function integrand
12
13  subroutine generate_sampling_data(f, x1, x2, y1, y2, N, filename)
14    procedure(math_func_2D) :: f
15    integer, intent(in) :: N
16    character(len=*), intent(in) :: filename
17    real(dp), intent(in) :: x1, x2, y1, y2
18    real(dp), allocatable :: x(:), y(:), z(:)
19    integer :: i
20
21    allocate(x(N), y(N), z(N))
22    call uniform(x1, x2, x)
23    call uniform(y1, y2, y)
24    z = f(x, y)
25    open(unit=10, file=filename, status='replace')
26    write(10, *) "# x          y          f(x, y)"
27
28    do i = 1, N
29      write(10, '(3F10.5)') x(i), y(i), z(i)
30    end do
31
32    close(10)
33    deallocate(x, y, z)
34  end subroutine generate_sampling_data
35 end module prob10_physics
36
37
38 program prob10_driver
39   use constants_mod
40   use monte_carlo_toolbox
41   use prob10_physics
42   implicit none
43   real(dp) :: x1, x2, y1, y2, exact, integral
44   integer :: N
45
46   N = 1000000
47
48   x1 = 0.0_dp
49   x2 = 1.0_dp
50   y1 = 0.0_dp
51   y2 = 1.0_dp
52   exact = 1.0_dp/3.0_dp
53
54   call random_seed()
55   integral = MCI_2D(integrand, x1, x2, y1, y2, N)

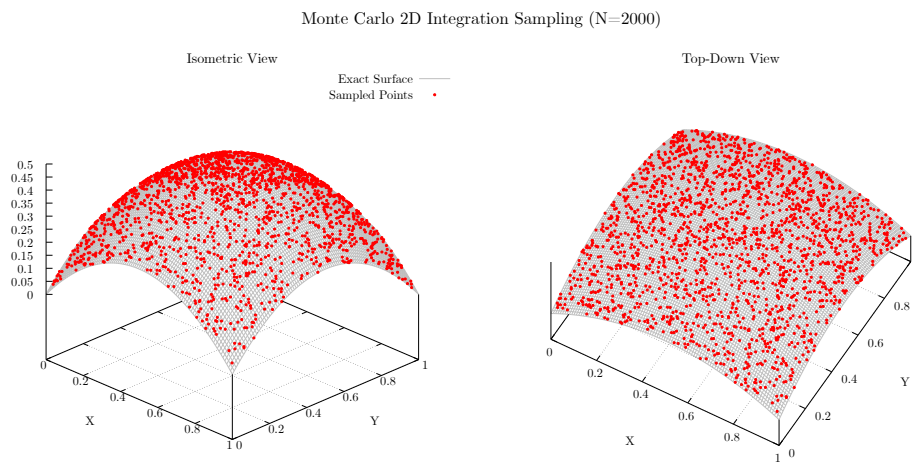
```

```

56
57 write(*, '(A)') "Function: f(x) = (x+y)-(x^2+y^2)"
58 write(*, '(A)') "-----"
59 write(*, '(A, G0.10)') "Estimated value: ", integral
60 write(*, '(A, G0.10)') "Actual value: ", exact
61
62 call generate_sampling_data(integrand, x1, x2, y1, y2, 2000, '
data/prob10_sampling.dat')
63
64 end program prob10_driver

```

- **Sampling Plot:** The function $f(x, y) = (x + y) - (x^2 + y^2)$ is plotted along with the randomly generated points (x_i, y_i) inside the 1×1 square grid using 3D plot. We expect the points will be on the plotted surface with a random distribution.



- **Output:**

```

Function: f(x) = (x+y)-(x^2+y^2)
-----
Estimated value: 0.3335578909
Actual value: 0.333333333

```

5 Remarks and Conclusion

Based on the computational experiments and subsequent data analysis performed in this study, several key observations regarding Monte Carlo integration methods can be made:

1. **Validation of the Central Limit Theorem:** The empirical results flawlessly reproduced theoretical expectations. Across all tested functions, the log-log curve fitting of the absolute error consistently yielded slopes of approximately -0.46 to -0.50 . This mathematically verifies that the algorithm's statistical error is fundamentally proportional to $\frac{1}{\sqrt{N}}$, confirming the robustness of the pseudorandom number generator used.

2. **Dimensional Independence and Efficiency:** While deterministic grid-based methods (like the Trapezoidal rule or Simpson's rule) are generally vastly superior for smooth 1D integrals, their computational cost explodes in higher dimensions—a phenomenon known as the "curse of dimensionality." The data in this report proves that the Monte Carlo error convergence rate of $O(N^{-0.5})$ remains completely unaffected when moving from a 1D curve to a 2D surface. This cements Monte Carlo as the most viable numerical method for multi-dimensional thermodynamic and quantum mechanical systems.
3. **Methodological Trade-offs:** The Hit-or-Miss method proved highly intuitive and exceptionally capable of handling complex, non-linear integration boundaries (such as the circular unit domain). However, it is inherently less efficient than the Mean Value method, as computational resources are "wasted" evaluating and discarding rejected points. The Mean Value method generally produces a lower absolute variance for the same number of samples as it uses every sample point to calculate a weighted average, making it the preferred algorithm when the boundaries are simple.
4. **The Cost of High Precision:** The $\frac{1}{\sqrt{N}}$ convergence rate, while reliable, is notoriously slow. To achieve just one additional decimal place of accuracy (reducing the error by a factor of 10), the number of computational iterations must increase by a factor of 100. Consequently, while Monte Carlo methods are brilliant for generating rapid approximations of complex multi-dimensional systems, they are highly computationally expensive if strictly high-precision exactness is required.